EECE 7205: Fundamentals of Computer Engineering

Karishma Premnath | Department of Electrical and Computer Engineering
Northeastern University | Fall 2026

Problem 1: Bubble Sort

1 Problem analysis

The task is to sort $A = \{22, 6, 4, 32, 7, 9, 12, 5\}$ into increasing order using Bubble Sort and to report how much work that takes. The function receives the array and its length $n = 8$ and rearranges it in place. The program prints the array after every pass, the final result, and totals for passes, comparisons and swaps.

Library sorting routines are not permitted, the program is built from `printArray` and `bubbleSort`, and no printing loop may appear outside `printArray`. The inputs I checked against are an array already in order, one reversed, one with repeated entries, and lengths of 0 and 1.

Question 5 wants a pass count for the sorted case, and neither required counter supplies it, so `bubbleSort` takes a third reference parameter: `bubbleSort(int arr[], int n, int &passes, int &comparisons, int &swaps)`.

2 Selected approach

The method rests on one observation. If a sweep always pushes the larger of two neighbors to the right, then whichever value is largest in the swept region finishes at that region's right edge. Repeating the sweep locks one more entry into place each time, working inward from the right, so the swept region can be one position shorter on every repetition.

A sweep that changes nothing is also informative, since it means no neighboring pair was out of order. I keep a boolean for that and leave the outer loop as soon as a sweep finishes with it unset.

3 Pseudocode

Algorithm 1 Bubble Sort with early exit and operation counters

Input: An array of numbers A. **Output:** Array A sorted in increasing order.

```
1:  n ← size(A)
2:  passes ← 0;  comparisons ← 0;  swaps ← 0
3:  for pass ← 0 to n − 2 do
4:      swapped ← False
5:      for i ← 0 to n − pass − 2 do
6:          comparisons ← comparisons + 1
7:          if A[i] > A[i + 1] then
8:              Swap(A[i], A[i + 1])
9:              swaps ← swaps + 1;  swapped ← True
10:         end if
11:     end for
12:     passes ← passes + 1
13:     print "Pass", passes;  PrintArray(A, n)
14:     if swapped = False then
15:         break
16:     end if
17: end for
18: return A
```

4 Hand trace on A

i	Comparison	Swap	Array afterwards
0	$22 > 6$	yes	6 22 4 32 7 9 12 5
1	$22 > 4$	yes	6 4 22 32 7 9 12 5
2	$22 > 32$	no	6 4 22 32 7 9 12 5
3	$32 > 7$	yes	6 4 22 7 32 9 12 5
4	$32 > 9$	yes	6 4 22 7 9 32 12 5
5	$32 > 12$	yes	6 4 22 7 9 12 32 5
6	$32 > 5$	yes	6 4 22 7 9 12 5 32

Table 1: Pass 1, comparison by comparison, from 22 6 4 32 7 9 12 5.

22 moves right past 6 and 4, stops when it meets 32, and 32 then carries on to the end. Seven comparisons and six exchanges place 32.

Pass	Comparisons	Swaps	Array after the pass
1	7	6	6 4 22 7 9 12 5 **32**
2	6	5	4 6 7 9 12 5 **22 32**
3	5	1	4 6 7 9 5 **12 22 32**
4	4	1	4 6 7 5 **9 12 22 32**
5	3	1	4 6 5 **7 9 12 22 32**
6	2	1	4 5 6 **7 9 12 22 32**
7	1	0	4 5 6 **7 9 12 22 32**
Total	**28**	**15**	

Table 2: All seven passes. Bold values are in their final positions.

This array needs every one of the $n - 1 = 7$ passes, so 28 comparisons is the most Bubble Sort can spend on eight elements. The flag does fire, but only at the end of pass 7, which the loop bound would have stopped anyway, so the early exit saves nothing here.

Each exchange of neighbors cancels one inversion, and A holds 15: 22 sits ahead of 6 smaller entries, 6 ahead of 2, 32 ahead of 4, and 7, 9 and 12 ahead of one each. That matches the 15 swaps in the trace.

5 Correctness explanation

Within one sweep, the value being carried right is replaced whenever it meets a larger one, so the value left at the far right of the swept region is always the largest in it. Sweep *pass* covers positions 0 through $n - 2 - pass$, so each repetition fixes one more position at the right end. By induction the array is ordered once everything but position 0 is settled, and that last value has nowhere else to go.

The loop terminates because *pass* increases every time and is bounded by $n - 2$. It can also stop sooner: a sweep with no exchange has found every neighboring pair in order, which is the same as the array being sorted.

Equal values fail the test $A[i] > A[i + 1]$, so they never move past each other and the sort is stable. For n of 0 or 1 the outer loop fails its first test and the array is returned unchanged.

6 Time and space complexity

Case	Exact count	Complexity
Best	$4n + 11$	$\Theta(n)$
Average	$\frac{11n^2+25n-8}{4}$	$\Theta(n^2)$
Worst	$\frac{7n^2+11n-4}{2}$	$\Theta(n^2)$
Auxiliary space	—	$\Theta(1)$

Table 3: Complexity of Bubble Sort, from a line-by-line frequency count.

The worst case is a reversed array, where all $n-1$ passes run and the inner body executes $\frac{n(n-1)}{2}$ times, which makes the total quadratic. The best case is a sorted array, where the first sweep exchanges nothing and the flag ends the loop, leaving $\Theta(n)$ work. Random input exchanges on roughly half the comparisons, which reduces part of the total but keeps it quadratic. Space is $\Theta(1)$, since the array is rearranged in place. For $n = 8$ these come to 43, 224 and 266 operations, and A used the full $\frac{n(n-1)}{2} = 28$ comparisons, so it is a worst case for comparisons even though its 15 exchanges are only a little over half of 28.

7 C++ program

```cpp
// ================================================================================
//Problem 1: Bubble Sort
// ================================================================================

#include <iostream>

using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];

        if (k < n - 1)
            cout << " ";
    }

    cout << endl;
}

void bubbleSort(int arr[], int n, int &passes, int &comparisons, int &swaps) {
    passes = 0;
    comparisons = 0;
    swaps = 0;

    for (int pass = 0; pass <= n - 2; pass++) {
        bool swapped = false;

        // The last "pass" values are already in place.
        for (int i = 0; i <= n - pass - 2; i++) {
            comparisons++;

            if (arr[i] > arr[i + 1]) {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;

                swaps++;
                swapped = true;
            }
        }

        passes++;

        cout << "Pass " << passes << ": ";
        printArray(arr, n);

        if (swapped == false)
            break;
    }
}

int main() {
    int A[] = {22, 6, 4, 32, 7, 9, 12, 5};
    int n = sizeof(A) / sizeof(A[0]);

    int passes;
    int comparisons;
    int swaps;

    cout << "=== Run 1: original array A ===" << endl;
    cout << "Original: ";
    printArray(A, n);

    bubbleSort(A, n, passes, comparisons, swaps);

    cout << "Sorted:   ";
    printArray(A, n);
    cout << "Passes: " << passes
         << "   Comparisons: " << comparisons
```

```
        << "   Swaps: " << swaps << endl;

    // A is sorted now, so B is the already sorted copy.
    int B[8];

    for (int k = 0; k < n; k++)
        B[k] = A[k];

    cout << endl << "=== Run 2: already sorted copy of A ===" << endl;
    cout << "Original: ";
    printArray(B, n);

    bubbleSort(B, n, passes, comparisons, swaps);

    cout << "Sorted:   ";
    printArray(B, n);
    cout << "Passes: " << passes
         << "   Comparisons: " << comparisons
         << "   Swaps: " << swaps << endl;

    return 0;
}
```

8 Execution screenshot

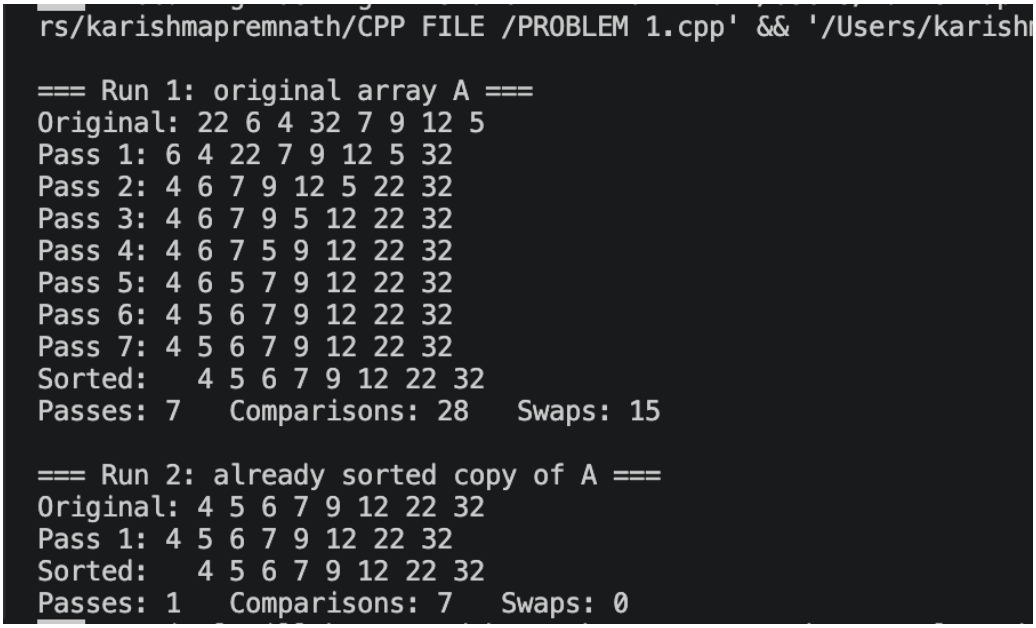


Figure 1: Program output. Run 1 sorts A, Run 2 sorts an already sorted copy.

Run 2 answers question 5. The ordered copy takes one pass and 7 comparisons with nothing exchanged, against 7 passes and 28 comparisons for A. No test in that pass succeeds, so `swapped` stays unset and the loop exits at the check. The $n-1$ comparisons are the price of confirming order. Without the flag all 7 passes would run on an already sorted array, making the best case $\Theta(n^2)$.

Problem 2: Insertion Sort

1 Problem analysis

The task is to sort $A = \{22, 6, 4, 32, 7, 9, 12, 5\}$ into increasing order using Insertion Sort, with $n = 8$. This is the array from Problem 1, so the shift count here can be set against the swap count there. The function receives the array and its length and reorders it in place. After each key is placed the program prints the step number, the key and the array, and at the end it reports the total comparisons and shifts.

A shift means one element moved a single position to the right, so placing one key can cause several shifts. That makes the shift count larger than a swap count would be for the same amount of work.

The sorting is done by `insertionSort` and every array print goes through `printArray`; no library sorting routine is used. A small `runCase` helper prints the label and the before and after arrays for one ordering, so the three runs do not repeat the same block of code. The counters are reference parameters, so the signature is `insertionSort(int arr[], int n, int &comparisons, int &shifts)`, and the totals are printed in `main()`. The last part of the question asks for two more orderings of the same eight values, so the program performs three runs and Run 1 works on the original unsorted array.

2 Selected approach

Insertion Sort keeps the left part of the array in order and grows it one element at a time. At step i the value at position i is lifted out and held as the key, leaving a gap. Working leftward, every element larger than the key is copied one place right, closing over the gap and opening a new one behind it. As soon as an element no greater than the key appears, or the left end is reached, the gap is exactly where the key belongs.

Because the left part is already ordered, the scan can stop at the first element that is not larger than the key. Everything further left is smaller still, so there is nothing left to check. That early stop is what makes the method cheap on input that is nearly sorted, and it is why the increasing ordering costs so little.

In both the pseudocode and the C++ the comparison is written out as an explicit test inside the loop body rather than left in the `while` condition, so that the counter catches the final failing test as well as the successful ones. The loop condition is then only the bounds check $pos \geq 0$, which is not a comparison between array values and is not counted.

3 Pseudocode

Algorithm 2 Insertion Sort with comparison and shift counters

Input: An array of numbers A. **Output:** Array A sorted in increasing order.

1: $comparisons \leftarrow 0$; $shifts \leftarrow 0$
2: **for** $i \leftarrow 1$ **to** $n - 1$ **do**
3: $key \leftarrow A[i]$
4: $pos \leftarrow i - 1$
5: **while** $pos \geq 0$ **do**
6: $comparisons \leftarrow comparisons + 1$
7: **if** $A[pos] \leq key$ **then**
8: **break** ▷ the key belongs just after pos
9: **end if**
10: $A[pos + 1] \leftarrow A[pos]$
11: $shifts \leftarrow shifts + 1$
12: $pos \leftarrow pos - 1$
13: **end while**
14: $A[pos + 1] \leftarrow key$
15: **print** "i = ", i, ", key = ", key; PrintArray(A, n)
16: **end for**
17: **return** A

4 Hand trace on A

Step $i = 7$ moves the most, so I traced its inner loop in full. The array entering that step is `4 6 7 9 12 22 32 5` and the key is 5, which sits at the far right and belongs in position 1.

pos	**Test**	**Shift**	**Array during the step**
6	$32 > 5$	yes	`4 6 7 9 12 22 32 32`
5	$22 > 5$	yes	`4 6 7 9 12 22 22 32`
4	$12 > 5$	yes	`4 6 7 9 12 12 22 32`
3	$9 > 5$	yes	`4 6 7 9 9 12 22 32`
2	$7 > 5$	yes	`4 6 7 7 9 12 22 32`
1	$6 > 5$	yes	`4 6 6 7 9 12 22 32`
0	$4 > 5$	no	`4 6 6 7 9 12 22 32`
the scan stops, so the key is written into position 1			`4 5 6 7 9 12 22 32`

Table 4: Inner loop of step $i = 7$, key = 5. Seven comparisons and six shifts. The last row is the failing test that ends the scan, which the counter includes.

The repeated values in the middle rows are the gap travelling right. Nothing is lost, because the key is held in its own variable until the gap reaches its final place.

i	**key**	**Comparisons**	**Shifts**	**Array after the key is inserted**
1	6	1	1	`6 22 4 32 7 9 12 5`
2	4	2	2	`4 6 22 32 7 9 12 5`
3	32	1	0	`4 6 22 32 7 9 12 5`
4	7	3	2	`4 6 7 22 32 9 12 5`
5	9	3	2	`4 6 7 9 22 32 12 5`
6	12	3	2	`4 6 7 9 12 22 32 5`
7	5	7	6	`4 5 6 7 9 12 22 32`
Total		**20**	**15**	

Table 5: All seven steps. Only step 3 costs a single comparison and no shifts, because 32 was already larger than everything to its left.

The shift total can be checked against the inversions in A, since moving one element past a smaller one removes exactly one out-of-order pair. Counting them: 22 precedes six smaller values, 6 precedes two, 32 precedes four, and 7, 9 and 12 each precede one. That is 15, which matches. Bubble Sort made 15 swaps on this same array in Problem 1, for the same reason: both methods only ever move a value past one smaller neighbour at a time, so both must pay once per out-of-order pair.

5 Correctness explanation

At the start of step i the first i entries are in increasing order. The step removes $A[i]$ into *key* and copies right every entry greater than it, halting at the first entry that is not. Since the region was already ordered, that halting point has only smaller values behind it, so writing the key into the gap leaves the first $i + 1$ entries ordered. The loop runs from $i = 1$ to $n - 1$, so the whole array is ordered on exit.

Both loops terminate. The outer counter increases each step and is bounded by $n - 1$. The inner one lowers *pos* by one on every iteration, and *pos* starts below i, so it reaches -1 at the latest.

An element equal to the key fails the test $A[pos] > key$ and stops the scan, so equal values never move past one another and the sort is stable. When n is 0 or 1 the outer loop never runs and the array is returned unchanged.

6 Time and space complexity

Case	Exact count	Complexity
Best	$11n - 5$	$\Theta(n)$
Average	$\frac{3n^2+13n-4}{2}$	$\Theta(n^2)$
Worst	$3n^2 + 5n - 2$	$\Theta(n^2)$
Auxiliary space	—	$\Theta(1)$

Table 6: Complexity of Insertion Sort, from a line-by-line frequency count.

The worst case is decreasing order, where every key travels the full width of the sorted region, so the inner loop runs $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ times and the total is quadratic. The best case is increasing order, where each key stops on its first comparison, giving $n - 1$ comparisons and no shifts. Random input places a key near the middle of the sorted region on average, which halves the inner work but does not change the order. Space is $\Theta(1)$, since only *key*, *pos* and the counters are extra and the array is rearranged in place. For $n = 8$ these come to 83, 146 and 230 operations.

7 C++ program

File name: `Assignment01_Problem02_Premnath.cpp`. Compiled with `g++ -std=c++17 -Wall -o problem2 Assignment01_Problem02_Premnath.cpp`.

```cpp
// ============================================================================
//Problem 2: Insertion Sort
// ============================================================================

#include <iostream>

using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];

        if (k < n - 1)
            cout << " ";
    }

    cout << endl;
}

void insertionSort(int arr[], int n, int &comparisons, int &shifts) {
    comparisons = 0;
    shifts = 0;

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int pos = i - 1;

        // The test sits here, not in the while condition, so the failing
        // comparison that ends the scan is counted too.
        while (pos >= 0) {
            comparisons++;

            if (arr[pos] <= key)
                break;

            arr[pos + 1] = arr[pos];
            shifts++;
            pos--;
        }

        arr[pos + 1] = key;

        cout << "i = " << i << ", key = " << key << ": ";
        printArray(arr, n);
    }
}

// Sorts one ordering and hands the totals back so main() can report them.
void runCase(const char *label, int arr[], int n, int &comparisons, int &shifts) {
    cout << label << endl;
```

```
    cout << "Original: ";
    printArray(arr, n);

    insertionSort(arr, n, comparisons, shifts);

    cout << "Sorted: ";
    printArray(arr, n);
}

int main() {
    int A[] = {22, 6, 4, 32, 7, 9, 12, 5};
    int up[] = {4, 5, 6, 7, 9, 12, 22, 32};
    int down[] = {32, 22, 12, 9, 7, 6, 5, 4};

    int n = sizeof(A) / sizeof(A[0]);

    int comparisons;
    int shifts;

    runCase("=== Run 1: array A ===", A, n, comparisons, shifts);
    cout << "Comparisons: " << comparisons << "   Shifts: " << shifts << endl;

    cout << endl;
    runCase("=== Run 2: increasing order (fewest shifts) ===", up, n,
            comparisons, shifts);
    cout << "Comparisons: " << comparisons << "   Shifts: " << shifts << endl;

    cout << endl;
    runCase("=== Run 3: decreasing order (most shifts) ===", down, n,
            comparisons, shifts);
    cout << "Comparisons: " << comparisons << "   Shifts: " << shifts << endl;

    return 0;
}
```

8 Execution screenshot

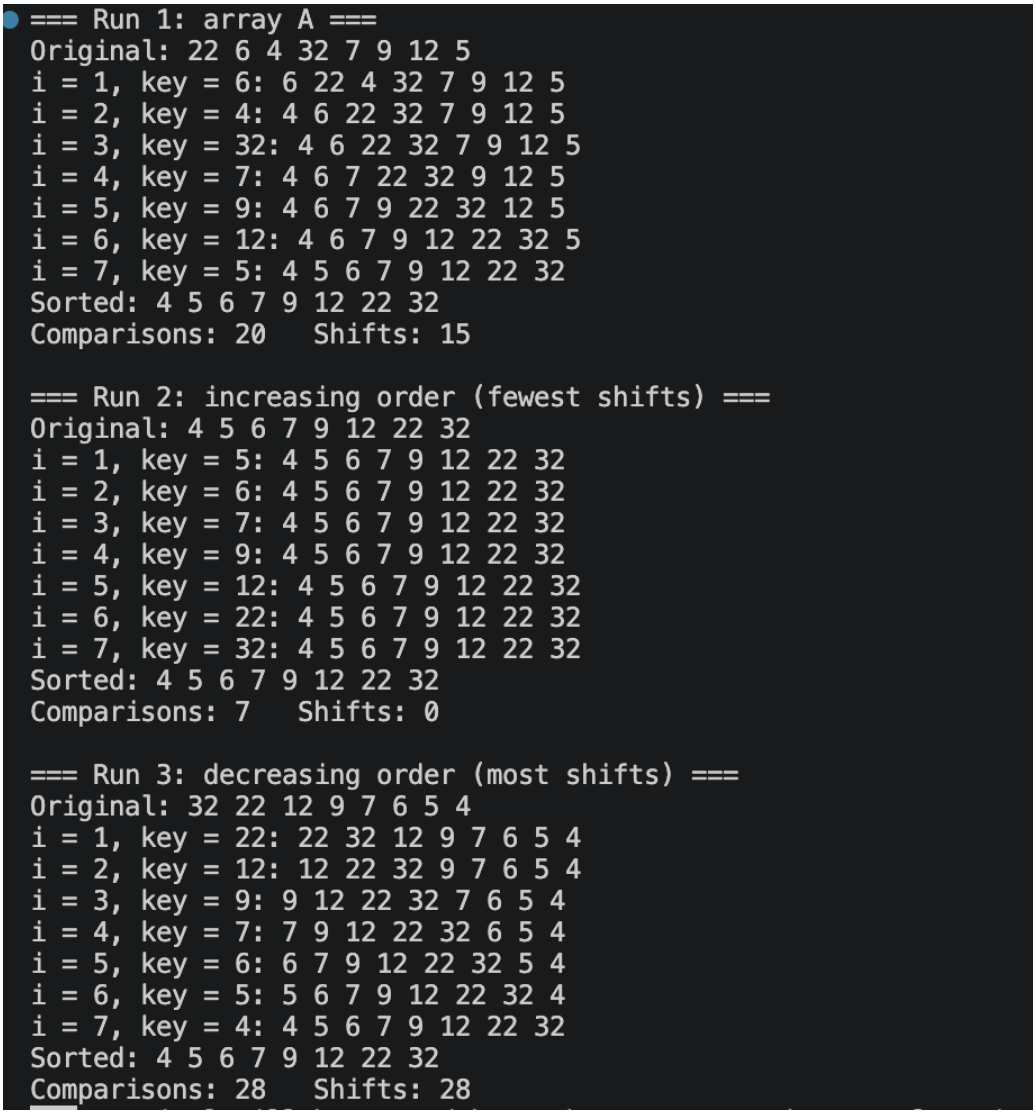


Figure 2: Program output. Run 1 sorts A, Runs 2 and 3 cover the last part of the question.

Ordering	**Array**	**Comparisons**	**Shifts**
As given	`22 6 4 32 7 9 12 5`	20	15
Increasing	`4 5 6 7 9 12 22 32`	7	0
Decreasing	`32 22 12 9 7 6 5 4`	28	28

Table 7: The same eight values in three orderings.

Increasing order gives the fewest shifts and decreasing order the most. A shift happens exactly when one value has to move past a smaller one, so the shift count equals the number of out-of-order pairs in the input. Increasing order has none, and every key stops on its first comparison, costing $n - 1 = 7$ comparisons and no shifts at all. Decreasing order has every one of the $\frac{n(n-1)}{2} = 28$ pairs out of order, so each key walks the whole sorted region and both measures reach 28. A itself lands between the two with 15 of those 28 pairs out of order. The single value 5 accounts for 6 of the 15, because it starts in the last position and belongs in the second.

Problem 3: Selection Sort

1 Problem analysis

Sort $A = \{22, 6, 4, 32, 7, 9, 12, 5\}$ into increasing order using Selection Sort, with $n = 8$. This is the array from Problem 1, because the last part of the question compares the swap counts of the two methods and that only means something on identical input.

Each pass prints the smallest value it found and the index it came from, then the array after the exchange. Totals for comparisons and swaps are printed in `main()`. The program uses `printArray` and `selectionSort`, no library sorting routine, and all array printing goes through `printArray`.

2 Selected approach

The array is held as a sorted front and an unsorted remainder. A pass scans the whole remainder, remembers where the smallest value sits, and exchanges it into the first unsorted slot, which grows the front by one. After $n - 1$ passes a single element is left and it must already be the largest.

The exchange is skipped whenever the smallest value is already in that slot, since swapping an element with itself achieves nothing. That test is the only way the input affects the amount of work done, because the scan visits every remaining element no matter what.

3 Pseudocode

Algorithm 3 Selection Sort with comparison and swap counters

Input: An array of numbers A. **Output:** Array A sorted in increasing order.

```
1: comparisons ← 0;  swaps ← 0
2: for i ← 0 to n − 2 do
3:     minPos ← i
4:     for j ← i + 1 to n − 1 do
5:         comparisons ← comparisons + 1
6:         if A[j] < A[minPos] then
7:             minPos ← j
8:         end if
9:     end for
10:    if minPos ≠ i then
11:        Swap(A[i], A[minPos]);  swaps ← swaps + 1
12:    end if
13:    print "Pass", i + 1, ": min =", A[i], "at index", minPos
14:    PrintArray(A, n)
15: end for
16: return A
```

4 Hand trace on A

j	Test	Outcome
1	$6 < 22$	smaller, $minPos = 1$
2	$4 < 6$	smaller, $minPos = 2$
3	$32 < 4$	no
4	$7 < 4$	no
5	$9 < 4$	no
6	$12 < 4$	no
7	$5 < 4$	no

Table 8: Pass 1 scan of `22 6 4 32 7 9 12 5`. Seven comparisons and two updates settle on 4 at index 2, which is then exchanged with 22.

Pass	Comparisons	Smallest	Index	Array after the pass
1	7	4	2	`4 6 22 32 7 9 12 5`
2	6	5	7	`4 5 22 32 7 9 12 6`
3	5	6	7	`4 5 6 32 7 9 12 22`
4	4	7	4	`4 5 6 7 32 9 12 22`
5	3	9	5	`4 5 6 7 9 32 12 22`
6	2	12	6	`4 5 6 7 9 12 32 22`
7	1	22	7	`4 5 6 7 9 12 22 32`
28 comparisons		**7 swaps**		

Table 9: All seven passes, with the sorted front in bold.

Every pass here found its minimum somewhere other than slot i, so the exchange ran each time and the swap total reaches its ceiling of $n-1 = 7$. The comparison count shrinks by one per pass, $7+6+5+4+3+2+1 = 28$, and would be the same for any input of this length.

5 Correctness explanation

Before pass i the first i slots hold the i smallest values in order. The pass looks at every remaining element and keeps the position of the smallest, so the exchange places that value in slot i and nothing outside the unsorted region moves. The first $i+1$ slots are then the $i+1$ smallest, in order. After $n-1$ passes only the largest value is unplaced, and the last slot is where it belongs.

Both loops are bounded by n and their counters only increase, so the method always finishes. Skipping the exchange when $minPos = i$ is safe, because in that case the value is already correct.

The scan uses a strict $<$, so an equal value never displaces the one already recorded. Even so the method is not stable, because the exchange can carry an element across the array and past an equal value further along.

6 Time and space complexity

Let r count the times a new smallest is recorded and s the exchanges. Every input costs the same $\frac{n(n-1)}{2}$ comparisons, so only r and s move:

$$T(n, r, s) = 2n^2 + 6n - 2 + r + 2s, \qquad 0 \le r \le \tfrac{n(n-1)}{2}, \quad 0 \le s \le n-1.$$

Case	Exact count	Complexity
Best ($r = s = 0$)	$2n^2 + 6n - 2$	$\Theta(n^2)$
Average	between the two bounds	$\Theta(n^2)$
Worst (upper bound)	$\frac{5n^2+15n-8}{2}$	$\Theta(n^2)$
Auxiliary space	—	$\Theta(1)$

Table 10: Complexity of Selection Sort, from a line-by-line frequency count.

All three cases are quadratic, which sets Selection Sort apart from Bubble Sort and Insertion Sort. Neither r nor s can rescue it, because the scan has no way to stop early: an element cannot be known to be smallest until every other one has been checked, so even an already sorted array costs the full $\frac{n(n-1)}{2}$ comparisons. Space is $\Theta(1)$. For $n = 8$ the best case is 174 operations and the upper bound 216, while A gave $r = 9$ and $s = 7$, so 197.

7 C++ program

File name: `Assignment01_Problem03_Premnath.cpp`. Compiled with `g++ -std=c++17 -Wall -o selection Assignment01_Problem03_Premnath.cpp`.

```cpp
// ============================================================================

// Problem 3: Selection Sort
// ============================================================================

#include <iostream>

using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];

        if (k < n - 1)
            cout << " ";
    }

    cout << endl;
}

void selectionSort(int arr[], int n, int &comparisons, int &swaps) {
    comparisons = 0;
    swaps = 0;

    for (int i = 0; i <= n - 2; i++) {
        int minPos = i;

        for (int j = i + 1; j <= n - 1; j++) {
            comparisons++;

            if (arr[j] < arr[minPos])
                minPos = j;
        }

        // Skipped when the smallest value already sits in slot i.
        if (minPos != i) {
            int temp = arr[i];
            arr[i] = arr[minPos];
            arr[minPos] = temp;

            swaps++;
        }

        cout << "Pass " << i + 1 << ": min = " << arr[i]
             << " at index " << minPos << " -> ";
        printArray(arr, n);
    }
}

int main() {
    int A[] = {22, 6, 4, 32, 7, 9, 12, 5};
    int n = sizeof(A) / sizeof(A[0]);

    int comparisons;
    int swaps;
```

```
56
57      cout << "Original: ";
58      printArray(A, n);
59
60      selectionSort(A, n, comparisons, swaps);
61
62      cout << "Sorted: ";
63      printArray(A, n);
64      cout << "Comparisons: " << comparisons << "   Swaps: " << swaps << endl;
65
66      return 0;
67  }
```

8 Execution screenshot

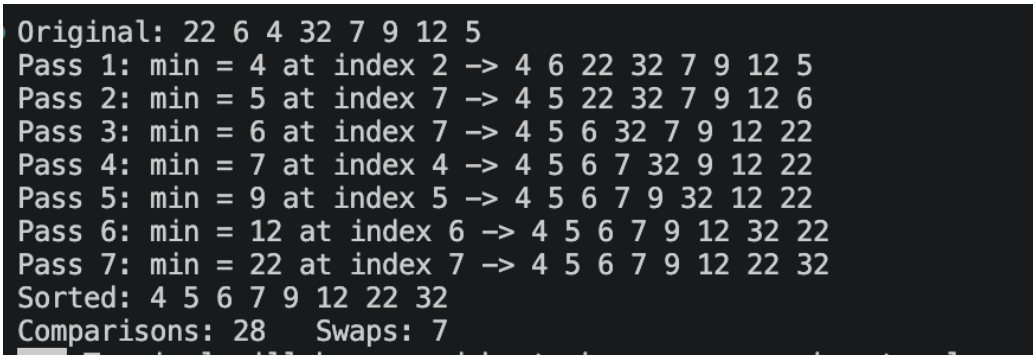


Figure 3: Program output for Selection Sort on A.

Algorithm	**Comparisons**	**Swaps**
Bubble Sort (Problem 1)	28	15
Selection Sort	28	7

Table 11: The two methods on the same array A.

Selection Sort uses 7 exchanges where Bubble Sort uses 15. The difference comes from how far a value is allowed to travel in one move. Selection Sort locates the smallest value first and then lifts it directly into place, so it can never exceed one exchange per pass and its total is capped at $n - 1$. Bubble Sort only ever exchanges neighbours, which means it needs one exchange for every out-of-order pair, and A holds 15 of those. Their comparison counts match at 28 because both examined every remaining pair, but that is specific to this input: Bubble Sort's flag can cut its comparisons short on nearly ordered data, whereas Selection Sort always pays the full $\frac{n(n-1)}{2}$.

Problem 4: Quick Sort

1 Problem analysis

Sort $A = \{22, 6, 4, 32, 7, 9, 12, 5\}$ using Quick Sort with the last element as pivot, Lomuto scheme, $n = 8$. The program prints the pivot and subarray at each recursive call, indented by depth so the output reads as a tree, traces the first partition with the values of i and j, and reports where the pivot lands. Comparisons are totalled in `main()`.

The sort is split between `quickSort` and a `partition` helper, which the question permits. Subarray printing uses the pointer form `printArray(arr + low, high - low + 1)`, so every array print still goes through `printArray`. A second run repeats the sort on an ordered copy.

2 Selected approach

Partitioning does all the work. The last element of a range becomes the pivot, and one sweep moves every value no larger than it to the left, tracked by a boundary index i. A closing exchange drops the pivot just past that boundary, which is its finished position, so the two sides can be sorted independently and never merged.

How well this performs is decided by the pivot alone. A pivot near the middle halves the range and gives about $\log n$ levels. A pivot at either extreme peels off a single element and gives n levels, which is what the last section examines.

3 Pseudocode

Algorithm 4 Quick Sort, Lomuto partition with the last element as pivot

Input: Array A, bounds low and $high$. **Output:** $A[low \ldots high]$ in increasing order.

```
 1: procedure QuickSort(A, low, high)
 2:     if low < high then
 3:         print "pivot =", A[high], and PrintArray(A + low, high − low + 1)        ▷ indented by depth
 4:         split ← Partition(A, low, high)
 5:         QuickSort(A, low, split − 1)
 6:         QuickSort(A, split + 1, high)
 7:     end if
 8: end procedure

 9: procedure Partition(A, low, high)
10:     pivot ← A[high]
11:     i ← low − 1
12:     for j ← low to high − 1 do                                  ▷ count one comparison per test
13:         if A[j] ≤ pivot then
14:             i ← i + 1
15:             Swap(A[i], A[j])
16:         end if
17:     end for
18:     Swap(A[i + 1], A[high])
19:     return i + 1
20: end procedure
```

4 Hand trace on A

First partition, whole array, pivot $A[7] = 5$, with i starting at -1.

j	**Test** $A[j] \le 5$	i	**Array after the step**
0	22, no	-1	`22 6 4 32 7 9 12 5`
1	6, no	-1	`22 6 4 32 7 9 12 5`
2	4, yes	0	`4 6 22 32 7 9 12 5`
3	32, no	0	`4 6 22 32 7 9 12 5`
4	7, no	0	`4 6 22 32 7 9 12 5`
5	9, no	0	`4 6 22 32 7 9 12 5`
6	12, no	0	`4 6 22 32 7 9 12 5`
Closing exchange puts the pivot at index 1			`4 5 22 32 7 9 12 6`

Table 12: Seven comparisons, one exchange inside the sweep. Only 4 is below the pivot, so the boundary advances once.

Depth	**Range**	**Pivot**	**Subarray**	**Comparisons**
0	$[0, 7]$	5	`22 6 4 32 7 9 12 5`	7
1	$[2, 7]$	6	`22 32 7 9 12 6`	5
2	$[3, 7]$	22	`32 7 9 12 22`	4
3	$[3, 5]$	12	`7 9 12`	2
4	$[3, 4]$	9	`7 9`	1
Total				**19**

Table 13: The recursion tree. Pivots 5 and 6 sit near the bottom of their ranges, so each leaves an empty left side and only the right side recurses.

5 Correctness explanation

The sweep keeps two facts true: $A[low \dots i]$ holds only values no larger than the pivot, and $A[i{+}1 \dots j{-}1]$ only larger ones. When $A[j]$ qualifies, advancing i and exchanging appends it to the first group while pushing a larger value into the second, so both facts survive. After the sweep the closing exchange sets the pivot between the groups, which is its place in the sorted order.

Since that position is final, the sides are sorted separately with no merge. Each recursive range is strictly shorter, because the pivot belongs to neither, so the depth is bounded and ranges of one or zero elements return at once.

Values equal to the pivot pass the $\le$ test and join the left group, which keeps the partition valid but can carry an equal value across the array. Quick Sort is therefore not stable.

6 Time and space complexity

Partitioning a range of m elements costs $m-1$ comparisons, so the total depends only on how the ranges divide.

Case	**Split**	**Recurrence**	**Complexity**
Best	even halves	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Average	random order	$T(n) = \Theta(n \log n)$	$\Theta(n \log n)$
Worst	one element off	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$

Table 14: Quick Sort rests entirely on the quality of the pivot.

In the worst case the comparisons sum to $(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$, which is 28 for $n = 8$. Array A needed 19, comfortably under that, because pivots 5 and 6 cut the range down quickly even though the tree leaned to one side.

Partitioning is in place, so the only extra storage is the recursion stack: $\Theta(\log n)$ when splits are even and $\Theta(n)$ when they are not. This run reached depth 5, the sorted run reached 7.

7 C++ program

```cpp
// ============================================================================

// Problem 4: Quick Sort
//
============================================================================

#include <iostream>

using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];

        if (k < n - 1)
            cout << " ";
    }

    cout << endl;
}

void indent(int depth) {
    for (int k = 0; k < depth; k++)
        cout << "  ";
}

int partition(int arr[], int low, int high, int &comparisons, bool trace) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        comparisons++;

        if (arr[j] <= pivot) {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;

            if (trace) {
                cout << "  i = " << i << ", j = " << j << " -> ";
                printArray(arr + low, high - low + 1);
            }
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    if (trace) {
        cout << "  pivot " << pivot << " lands at index " << i + 1 << " -> ";
        printArray(arr + low, high - low + 1);
    }

    return i + 1;
}

void quickSort(int arr[], int low, int high, int depth, int &comparisons) {
    if (low >= high)
        return;

    indent(depth);
    cout << "pivot = " << arr[high] << ": ";
    printArray(arr + low, high - low + 1);

    int split = partition(arr, low, high, comparisons, depth == 0);

    quickSort(arr, low, split - 1, depth + 1, comparisons);
    quickSort(arr, split + 1, high, depth + 1, comparisons);
}

int main() {
    int A[] = {22, 6, 4, 32, 7, 9, 12, 5};
    int B[] = {4, 5, 6, 7, 9, 12, 22, 32};
    int n = sizeof(A) / sizeof(A[0]);

    int comparisons = 0;

    cout << "=== Run 1: array A ===" << endl;
    cout << "Original: ";
```

```
    printArray(A, n);

    quickSort(A, 0, n - 1, 0, comparisons);

    cout << "Sorted: ";
    printArray(A, n);
    cout << "Comparisons: " << comparisons << endl;

    comparisons = 0;

    cout << endl << "=== Run 2: the same values already sorted ===" << endl;
    cout << "Original: ";
    printArray(B, n);

    quickSort(B, 0, n - 1, 0, comparisons);

    cout << "Sorted: ";
    printArray(B, n);
    cout << "Comparisons: " << comparisons << endl;

    return 0;
}
```

8 Execution screenshot

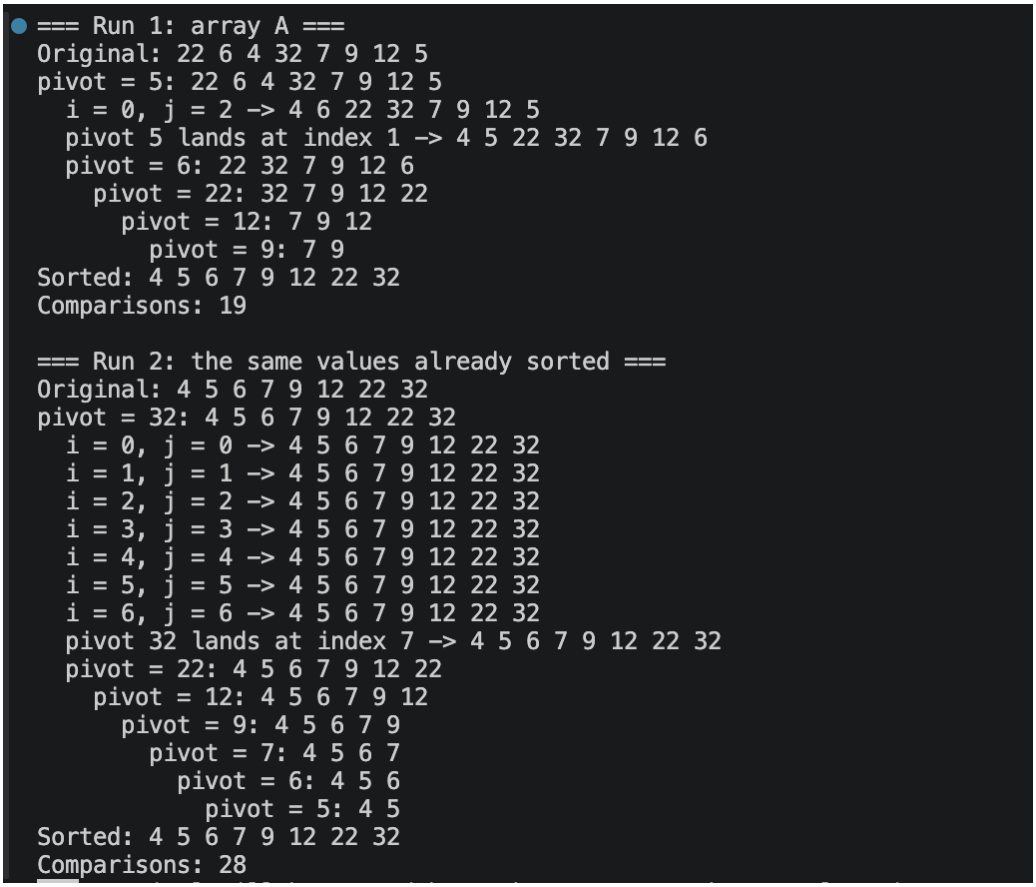


Figure 4: Program output. Run 1 sorts A, Run 2 sorts an ordered copy.

Input	Comparisons	Levels	Tree shape
A as given	19	5	branching
`4 5 6 7 9 12 22 32`	28	7	a single chain

Table 15: The same values, unsorted and sorted.

The ordered copy costs 28 comparisons against 19, and the printed tree collapses into one long chain. The pivot rule is the cause: in an ordered range the last element is the largest present, so every test $A[j] \leq pivot$ succeeds, the boundary runs the full width, and the pivot returns to the right-hand end. That leaves $m - 1$ elements on the left and none on the right, so nothing is divided. The number of levels becomes $n - 1$ instead of $\log n$ and the comparisons sum to $\frac{n(n-1)}{2} = 28$. Ordered data is what this rule handles worst, which matters because real input is often partly ordered already.

The fix is to stop letting position choose the pivot. Taking the median of the first, middle and last elements and exchanging it into the last slot costs two comparisons per call and makes an ordered range split evenly, since its middle element is its median. Picking a random index and exchanging it to the end also works and leaves no single input that reliably triggers the worst case.

Problem 5: Merge Sort

1 Problem analysis

Sort $A = \{22, 6, 4, 32, 7, 9, 12, 5\}$ with Merge Sort, $mid = (low + high)/2$, $n = 8$. Each subarray is printed when split and again after merging, and the program counts the comparisons made while merging, reporting how many fall in the final merge. It is built from `printArray` and `mergeSort` with a `merge` helper, which the question permits, and all printing goes through `printArray`. Runs 2 and 3 use a sorted copy and a reversed copy.

2 Selected approach

The array is halved by index until each piece holds one element, then pieces are merged in pairs by walking both fronts and copying out whichever value is smaller; once a side empties the rest of the other is copied with no further comparisons. The split ignores the values, so the recursion has the same shape for any input of length n. Merging needs temporary arrays to stay linear, so this is the one algorithm here that does not sort in place.

3 Pseudocode

Algorithm 5 Merge Sort with a comparison counter

Input: Array A, bounds low and $high$. **Output:** $A[low \ldots high]$ in increasing order.

```
 1: procedure MergeSort(A, low, high)
 2:     if low < high then
 3:         print "split:";  PrintArray(A + low, high − low + 1)
 4:         mid ← (low + high)/2
 5:         MergeSort(A, low, mid)
 6:         MergeSort(A, mid + 1, high)
 7:         Merge(A, low, mid, high)
 8:     end if
 9: end procedure

10: procedure Merge(A, low, mid, high)
11:     L ← copy of A[low ... mid]
12:     R ← copy of A[mid + 1 ... high]
13:     i ← 0;  j ← 0;  k ← low
14:     while i < |L| and j < |R| do
15:         if L[i] ≤ R[j] then
16:             A[k] ← L[i];  i ← i + 1
17:         else
18:             A[k] ← R[j];  j ← j + 1
19:         end if
20:         k ← k + 1
21:     end while
22:     copy whatever is left of L into A from k
23:     copy whatever is left of R into A from k
24:     print "merged:";  PrintArray(A + low, high − low + 1)
25: end procedure
```

4 Hand trace on A

Step	Comparison	Taken	Output so far
1	$4 \le 5$	4	`4`
2	$6 \le 5$? no	5	`4 5`
3	$6 \le 7$	6	`4 5 6`
4	$22 \le 7$? no	7	`4 5 6 7`
5	$22 \le 9$? no	9	`4 5 6 7 9`
6	$22 \le 12$? no	12	`4 5 6 7 9 12`
right side empty, copy 22 and 32			`4 5 6 7 9 12 22 32`

Table 16: Final merge of `4 6 22 32` with `5 7 9 12`.

Merge	Result	Comparisons
`22` with `6`	`6 22`	1
`4` with `32`	`4 32`	1
`6 22` with `4 32`	`4 6 22 32`	3
`7` with `9`	`7 9`	1
`12` with `5`	`5 12`	1
`7 9` with `5 12`	`5 7 9 12`	3
`4 6 22 32` with `5 7 9 12`	`4 5 6 7 9 12 22 32`	6
Total		**16**

Table 17: All seven merges. Six of the sixteen comparisons fall in the last one.

5 Correctness explanation

A single element is ordered. If both halves come back ordered, the merge always copies the smaller of the two front values, so the output grows in order, and when one side empties the rest of the other is already ordered and no smaller than anything written. Each range is strictly shorter than its parent, since $low \le mid < high$, so the recursion reaches single elements and stops. The test uses $\le$, so a tie goes to the left half and equal values keep their original order, which makes Merge Sort stable.

6 Time and space complexity

Case	Exact count	Complexity
Best	$\frac{11}{2} n \lg n + 16n - 15$	$\Theta(n \log n)$
Average	same recurrence	$\Theta(n \log n)$
Worst	$6n \lg n + 15n - 14$	$\Theta(n \log n)$
Auxiliary space	L and R, plus the stack	$\Theta(n) + \Theta(\log n)$

Table 18: Complexity of Merge Sort, counted line by line from the pseudocode.

There are $\lg n$ levels and each merges all n elements, so $T(n) = 2T(n/2) + \Theta(n)$ holds whatever the input and all three cases are $\Theta(n \log n)$. Only the comparisons move: a merge where neither half empties early costs $m - 1$, giving 17 for $n = 8$, while one where a half is copied out first costs $\frac{m}{2}$, giving 12. There is no early exit, so even a sorted array is split all the way down and merged at every level. Space is $\Theta(n)$ for the two temporary arrays plus $\Theta(\log n)$ for the recursion stack, the only algorithm here that does not sort in place.

7 C++ program

```cpp
// ============================================================================

// Problem 5: Merge Sort
//============================================================================

#include <iostream>

using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];

        if (k < n - 1)
            cout << " ";
    }

    cout << endl;
}

void merge(int arr[], int low, int mid, int high, int &comparisons, int &lastMerge) {
    int before = comparisons;
    int leftSize = mid - low + 1;
    int rightSize = high - mid;
    int left[16], right[16];

    for (int k = 0; k < leftSize; k++)
        left[k] = arr[low + k];

    for (int k = 0; k < rightSize; k++)
        right[k] = arr[mid + 1 + k];

    int i = 0, j = 0, k = low;

    while (i < leftSize && j < rightSize) {
        comparisons++;

        if (left[i] <= right[j])
            arr[k] = left[i++];
        else
            arr[k] = right[j++];

        k++;
    }

    while (i < leftSize)
        arr[k++] = left[i++];

    while (j < rightSize)
        arr[k++] = right[j++];

    lastMerge = comparisons - before;

    cout << "merged: ";
    printArray(arr + low, high - low + 1);
}

void mergeSort(int arr[], int low, int high, int &comparisons, int &lastMerge) {
    if (low >= high)
        return;

    cout << "split: ";
    printArray(arr + low, high - low + 1);

    int mid = (low + high) / 2;

    mergeSort(arr, low, mid, comparisons, lastMerge);
    mergeSort(arr, mid + 1, high, comparisons, lastMerge);
    merge(arr, low, mid, high, comparisons, lastMerge);
}

int main() {
    int A[] = {22, 6, 4, 32, 7, 9, 12, 5};
    int up[] = {4, 5, 6, 7, 9, 12, 22, 32};
    int down[] = {32, 22, 12, 9, 7, 6, 5, 4};
    int n = sizeof(A) / sizeof(A[0]);

    int comparisons = 0, lastMerge = 0;

```

```
80      cout << "=== Run 1: array A ===" << endl << "Original: ";
81      printArray(A, n);
82      mergeSort(A, 0, n - 1, comparisons, lastMerge);
83      cout << "Sorted: ";
84      printArray(A, n);
85      cout << "Comparisons: " << comparisons
86           << " (final merge: " << lastMerge << ")" << endl;
87
88      comparisons = 0;
89
90      cout << endl << "=== Run 2: sorted copy ===" << endl << "Original: ";
91      printArray(up, n);
92      mergeSort(up, 0, n - 1, comparisons, lastMerge);
93      cout << "Sorted: ";
94      printArray(up, n);
95      cout << "Comparisons: " << comparisons
96           << " (final merge: " << lastMerge << ")" << endl;
97
98      comparisons = 0;
99
100     cout << endl << "=== Run 3: reversed copy ===" << endl << "Original: ";
101     printArray(down, n);
102     mergeSort(down, 0, n - 1, comparisons, lastMerge);
103     cout << "Sorted: ";
104     printArray(down, n);
105     cout << "Comparisons: " << comparisons
106          << " (final merge: " << lastMerge << ")" << endl;
107
108     return 0;
109 }
```

8 Execution screenshot

```
=== Run 1: array A ===
Original: 22 6 4 32 7 9 12 5
split: 22 6 4 32 7 9 12 5
split: 22 6 4 32
split: 22 6
merged: 6 22
split: 4 32
merged: 4 32
merged: 4 6 22 32
split: 7 9 12 5
split: 7 9
merged: 7 9
split: 12 5
merged: 5 12
merged: 5 7 9 12
merged: 4 5 6 7 9 12 22 32
Sorted: 4 5 6 7 9 12 22 32
Comparisons: 16 (final merge: 6)

=== Run 2: sorted copy ===
Original: 4 5 6 7 9 12 22 32
split: 4 5 6 7 9 12 22 32
split: 4 5 6 7
split: 4 5
merged: 4 5
split: 6 7
merged: 6 7
merged: 4 5 6 7
split: 9 12 22 32
split: 9 12
merged: 9 12
split: 22 32
merged: 22 32
merged: 9 12 22 32
merged: 4 5 6 7 9 12 22 32
Sorted: 4 5 6 7 9 12 22 32
Comparisons: 12 (final merge: 4)

=== Run 3: reversed copy ===
Original: 32 22 12 9 7 6 5 4
split: 32 22 12 9 7 6 5 4
split: 32 22 12 9
split: 32 22
merged: 22 32
split: 12 9
merged: 9 12
merged: 9 12 22 32
split: 7 6 5 4
split: 7 6
merged: 6 7
split: 5 4
merged: 4 5
merged: 4 5 6 7
merged: 4 5 6 7 9 12 22 32
Sorted: 4 5 6 7 9 12 22 32
Comparisons: 12 (final merge: 4)
```

Figure 5: Program output for the three inputs.

Algorithm	A	**Sorted**	**Reversed**	**Spread**
Bubble Sort	28	7	28	21
Quick Sort	19	28	28	9
Merge Sort	16	12	12	**4**

Table 19: Comparisons on the same eight values in three orderings.

Merge Sort stays between 12 and 16 while Bubble Sort swings from 7 to 28. The split uses indices rather than values, so all three inputs give the same seven merges on the same subarray sizes; only the merging responds to the data, and a merge of p and q elements is confined between $\min(p, q)$ and $p + q - 1$. Both ordered copies land on the floor of 12, because in every merge one side drains before the other starts. Bubble Sort's flag and Quick Sort's pivot both depend on values, so their structure changes with the input: one pass against seven, a branching tree against a chain. Merge Sort gives up that sensitivity and pays $\Theta(n)$ extra memory for it, while every other algorithm here sorts in place.

Problem 6: Heap Sort

1 Problem analysis

Sort $A = \{22, 6, 4, 32, 7, 9, 12, 5\}$ using Heap Sort, $n = 8$. The array is read as a binary tree in which the children of index k sit at $2k + 1$ and $2k + 2$. The program prints the heap once the build phase finishes, then after each pass prints the heap part and the sorted part separately, and reports the total swaps made inside `siftDown`. It is built from `printArray` and `heapSort` with a `siftDown` helper, which the question permits, and all printing goes through `printArray`.

2 Selected approach

A max-heap keeps every parent at least as large as its children, so the largest value is always at index 0. The array is turned into one by pushing each non-leaf value down into place, starting from the last parent at $n/2-1$ and working back to the root. Leaves need no work, which is why the build skips the second half of the array.

Sorting then repeats one step: exchange the root with the last unsorted slot, which puts the current maximum in its final position, shrink the heap by one, and push the new root down to restore the ordering. After $n - 1$ steps the sorted region covers everything.

3 Pseudocode

Algorithm 6 Heap Sort with a swap counter

Input: Array A of size n. **Output:** A sorted in increasing order.

```
 1: procedure HeapSort(A, n)
 2:     for i ← n/2 − 1 down to 0 do
 3:         SiftDown(A, i, n)
 4:     end for
 5:     print "Heap:";  PrintArray(A, n)
 6:     for end ← n − 1 down to 1 do
 7:         Swap(A[0], A[end])
 8:         SiftDown(A, 0, end)
 9:         print heap part PrintArray(A, end)
10:         print sorted part PrintArray(A + end, n − end)
11:     end for
12: end procedure

13: procedure SiftDown(A, root, size)
14:     loop
15:         largest ← root
16:         if 2 root + 1 < size and A[2 root + 1] > A[largest] then
17:             largest ← 2 root + 1
18:         end if
19:         if 2 root + 2 < size and A[2 root + 2] > A[largest] then
20:             largest ← 2 root + 2
21:         end if
22:         if largest = root then
23:             return
24:         end if
25:         Swap(A[root], A[largest]);  swaps ← swaps + 1
26:         root ← largest
27:     end loop
28: end procedure
```

4 Hand trace on A

i	Action	Array after
3	6 is already above its child 5, nothing moves	`22 6 4 32 7 9 12 5`
2	4 falls past 12	`22 6 12 32 7 9 4 5`
1	6 falls past 32	`22 32 12 6 7 9 4 5`
0	22 falls past 32	`32 22 12 6 7 9 4 5`

Table 20: Build phase, three swaps. The loop starts at index $n/2 - 1 = 3$, because indices 4 to 7 are leaves.

end	Heap part	Sorted part
7	`22 7 12 6 5 9 4`	`32`
6	`12 7 9 6 5 4`	`22 32`
5	`9 7 4 6 5`	`12 22 32`
4	`7 6 4 5`	`9 12 22 32`
3	`6 5 4`	`7 9 12 22 32`
2	`5 4`	`6 7 9 12 22 32`
1	`4`	`5 6 7 9 12 22 32`

Table 21: Sorting phase. The heap shrinks by one each pass as the largest remaining value moves to the front of the sorted region.

The build phase made 3 swaps inside `siftDown` and the sorting phase made 9, giving 12 in total. The seven exchanges of the root with the last slot are not included, since the question asks only for the swaps inside `siftDown`.

5 Correctness explanation

SiftDown is called on a node whose two subtrees are already heaps. If the node is at least as large as both children the subtree is a heap and it stops; otherwise it exchanges with the larger child, which fixes the order at that node and can only break it one level down, where the process repeats. Since the index at least doubles each time, the loop ends at a leaf.

The build loop runs from the last parent back to the root, so whenever SiftDown is called both subtrees below it are already heaps, and the whole array is a heap when it finishes. In the sorting phase the root holds the largest remaining value, so moving it to position *end* puts it where it belongs; everything to the right of *end* is already sorted and no smaller. Shrinking the heap and sifting the new root down restores the heap on what is left, so after $n - 1$ passes the array is ordered.

Heap Sort is **not stable**. Take $\{2_a, 2_b, 1\}$, writing 2_a for the first copy. The build phase changes nothing, since neither child exceeds its parent. The first pass exchanges the root with the last slot, giving $\{1, 2_b, 2_a\}$, and SiftDown lifts 2_b back to the root. The second pass leaves $\{1, 2_b, 2_a\}$, so 2_b now comes before 2_a and the two equal values have traded places. The cause is that an exchange moves a value across the array rather than past its immediate neighbour, so nothing preserves the original relative order.

6 Time and space complexity

Phase / case	Work	Complexity
Build heap	$\sum$ over levels of nodes $\times$ height	$\Theta(n)$
Best case	$n - 1$ sift-downs of height $\lg n$	$\Theta(n \log n)$
Average case	same	$\Theta(n \log n)$
Worst case	same	$\Theta(n \log n)$
Auxiliary space	a few index variables, no recursion	$\Theta(1)$

Table 22: Complexity of Heap Sort.

The build phase is $\Theta(n)$ rather than $\Theta(n \log n)$, because half the nodes are leaves with no work, a quarter sink at most one level, an eighth at most two, and that series converges. The sorting phase dominates: each of the $n - 1$ passes sifts a value down a tree of height at most $\lg n$, giving $\Theta(n \log n)$. There is no early exit and no input-dependent shortcut, so best, average and worst agree, much as in Selection Sort but at $\Theta(n \log n)$ instead of $\Theta(n^2)$. Space is $\Theta(1)$: the heap lives inside the array, and SiftDown is written as a loop rather than a recursive call, so there is no stack to count.

7 C++ program

```cpp
// ============================================================================
// Problem 6: Heap Sort
//============================================================================

#include <iostream>

using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];

        if (k < n - 1)
            cout << " ";
    }

    cout << endl;
}

void siftDown(int arr[], int root, int size, int &swaps) {
    while (true) {
        int largest = root;
        int left = 2 * root + 1;
        int right = 2 * root + 2;

        if (left < size && arr[left] > arr[largest])
            largest = left;

        if (right < size && arr[right] > arr[largest])
            largest = right;

        if (largest == root)
            return;

        int temp = arr[root];
        arr[root] = arr[largest];
        arr[largest] = temp;

        swaps++;
        root = largest;
    }
}

void heapSort(int arr[], int n, int &swaps) {
    swaps = 0;


    for (int i = n / 2 - 1; i >= 0; i--)
        siftDown(arr, i, n, swaps);

    cout << "Heap: ";
    printArray(arr, n);

    // Sorting phase: move the root to the end, shrink, restore the heap.
    for (int end = n - 1; end >= 1; end--) {
        int temp = arr[0];
        arr[0] = arr[end];
        arr[end] = temp;

        siftDown(arr, 0, end, swaps);

        cout << "end = " << end << ": heap: ";
        printArray(arr, end);
        cout << "          sorted: ";
        printArray(arr + end, n - end);
    }
}

int main() {
```

```
    int A[] = {22, 6, 4, 32, 7, 9, 12, 5};
    int n = sizeof(A) / sizeof(A[0]);

    int swaps;

    cout << "Original: ";
    printArray(A, n);

    heapSort(A, n, swaps);

    cout << "Sorted: ";
    printArray(A, n);
    cout << "Swaps in siftDown: " << swaps << endl;

    return 0;
}
```

8 Execution screenshot

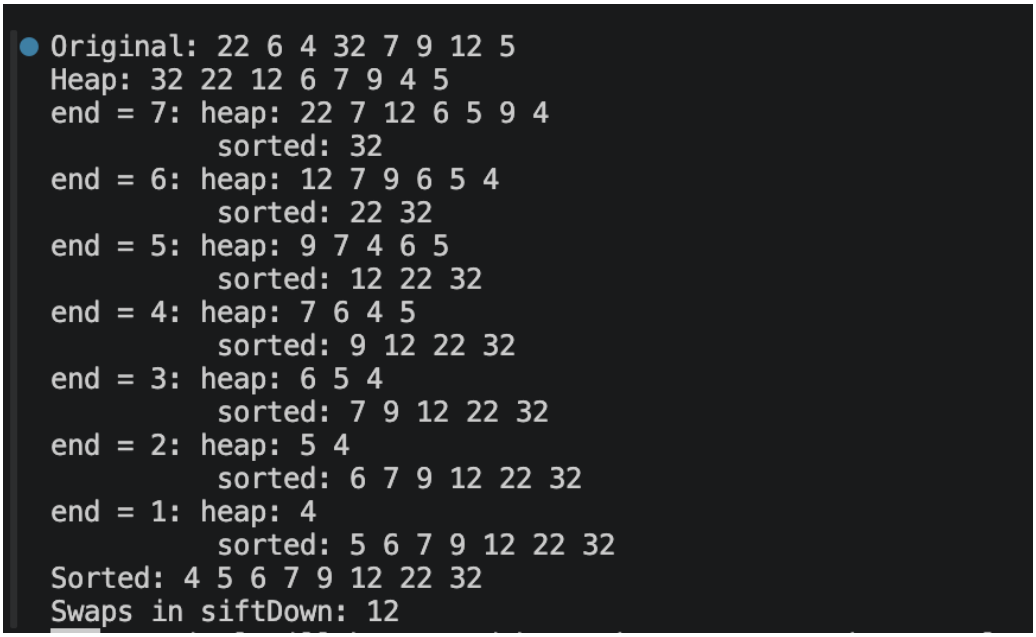


Figure 6: Program output. The heap after the build phase, then each pass with its heap part and sorted part.

The heap printed after the build phase, `32 22 12 6 7 9 4 5`, matches the tree drawn in Section 4, and the seven passes match the table beside it. The 12 swaps divide into 3 from the build phase and 9 from the sorting phase.

Problem 7: Comparing the Six Algorithms

1 Problem analysis

Put all six sorts from Problems 1 to 6 into one program with their step-by-step printing removed, then measure them. Three measurements are asked for: the comparison count of each algorithm on A, on A sorted and on A reversed; the running time of each on random arrays of 1,000, 5,000 and 10,000 values drawn from 1 to 100,000; and the first ten elements of each sorted array as a correctness check.

There is no new algorithm here, so the pseudocode below covers only the measurement. The six sorts are unchanged apart from the printing being taken out and a comparison counter added, so their pseudocode is not repeated.

2 Method

Every algorithm must see the same input, so the data is generated once and copied into a working array before each sort. Without that step the later algorithms would receive an array the earlier ones had already ordered.

`srand(1)` fixes the seed, so the same values are used at each size and the run can be repeated. Timing uses `high_resolution_clock` from `<chrono>`, read immediately before and after the sort call so the copying is not counted. Heap Sort counts the comparisons inside `siftDown`, the two tests against the left and right children, rather than the swaps that Problem 6 reported.

3 Pseudocode

Algorithm 7 Measurement harness for the six algorithms

Input: The array A and the sizes to time. **Output:** Comparison counts, running times, and a correctness check.

$A \leftarrow \{22, 6, 4, 32, 7, 9, 12, 5\}$; $up \leftarrow A$ in order; $down \leftarrow A$ reversed
for each algorithm s **do**
    for each input I in $\{A, up, down\}$ **do**
        copy I into $work$
        print RunSort$(s, work, 8)$
    end for
end for

for each size m in $\{1000, 5000, 10000\}$ **do**
    fill big with m random values from 1 to 100000
    for each algorithm s **do**
        copy big into $work$ ▷ same data for everyone
        $start \leftarrow$ clock
        RunSort$(s, work, m)$
        $stop \leftarrow$ clock
        $time[s][m] \leftarrow stop - start$
    end for
end for
print the time table

for each algorithm s **do**
    copy big into $work$; RunSort$(s, work, 10000)$
    PrintArray$(work, 10)$
end for

procedure RunSort$(s, work, n)$
    run the sort named by s on $work[0 \ldots n-1]$, counting comparisons
    return the comparison count
end procedure

4 Comparison counts

Algorithm	A	Sorted	Reversed
Bubble Sort	28	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	19	28	28
Merge Sort	16	12	12
Heap Sort	25	27	24

Table 23: Comparisons on the eight values of A in three orderings.

Bubble and Insertion Sort drop to 7 on sorted input, because both stop once they find everything in order. Selection Sort stays at 28 in all three columns, since it has no early exit and always scans the full unsorted region. Quick Sort is the only one that does worse on ordered input, 19 against 28, because a last-element pivot divides nothing when the values already ascend.

5 Running time

Algorithm	n = 1000	n = 5000	n = 10000	5k → 10k
Bubble Sort	3073	41462	150450	3.63
Insertion Sort	993	14066	57964	4.12
Selection Sort	1751	23432	95261	4.07
Quick Sort	110	579	1282	2.21
Merge Sort	159	780	1680	2.15
Heap Sort	190	883	1923	2.18

Table 24: Microseconds on random arrays, with the same data given to every algorithm. The last column is the factor by which the time grew when n doubled.

6 Correctness check

All six produced the same first ten values, `10 11 15 26 27 28 34 38 48 51`, from the same random input of 10,000 elements, so all six sorted correctly.

7 Discussion

The first three algorithms are quadratic and the last three are $n \log n$. Bubble, Insertion and Selection Sort each run two nested loops across the array, while Quick, Merge and Heap Sort cut the problem roughly in half at every step.

The timings agree. Doubling n from 5,000 to 10,000 should multiply a quadratic time by 4 and an $n \log n$ time by about 2.16; the measured factors were 3.63, 4.12 and 4.07 against 2.21, 2.15 and 2.18. These are single runs, so a few percent of scatter is expected. The gap says more than the ratios: Bubble Sort is 28 times slower than Quick Sort at 1,000 elements and 117 times slower at 10,000.

For a large random array I would use Quick Sort, which was fastest at all three sizes because it partitions in place and copies nothing. Its $\Theta(n^2)$ worst case needs input close to sorted order, which random data is not, but Merge Sort is the safer choice if that case must be ruled out.

For an almost sorted array I would use Insertion Sort, since each key stops at its first comparison when already in place: 7 comparisons on the sorted copy against 20 on A. Quick Sort would be the worst choice, because ordered input is what makes a last-element pivot split nothing.

8 C++ program

```cpp
// ========================================================================
// Problem 7: Comparing the Six Algorithms
//
// The six sorts from Problems 1 to 6 with their step-by-step printing removed,
// each counting comparisons. Part 1 compares counts, Part 2 measures time.
// ========================================================================

#include <iostream>
#include <chrono>
#include <cstdlib>

using namespace std;
using namespace std::chrono;

// scratch space for merge, big enough for the largest test
int mergeLeft[10000];
int mergeRight[10000];

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];

        if (k < n - 1)
            cout << " ";
    }

    cout << endl;
}

void bubbleSort(int arr[], int n, int &comparisons) {
    comparisons = 0;

    for (int pass = 0; pass <= n - 2; pass++) {
        bool swapped = false;

        for (int i = 0; i <= n - pass - 2; i++) {
            comparisons++;

            if (arr[i] > arr[i + 1]) {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                swapped = true;
            }
        }

        if (swapped == false)
            break;
    }
}

void insertionSort(int arr[], int n, int &comparisons) {
    comparisons = 0;

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int pos = i - 1;

        while (pos >= 0) {
            comparisons++;

            if (arr[pos] <= key)
                break;

            arr[pos + 1] = arr[pos];
            pos--;
        }

        arr[pos + 1] = key;
    }
}

void selectionSort(int arr[], int n, int &comparisons) {
    comparisons = 0;

    for (int i = 0; i <= n - 2; i++) {
        int minPos = i;

        for (int j = i + 1; j <= n - 1; j++) {
```

```
            comparisons++;

            if (arr[j] < arr[minPos])
                minPos = j;
        }

        if (minPos != i) {
            int temp = arr[i];
            arr[i] = arr[minPos];
            arr[minPos] = temp;
        }
    }
}

int partition(int arr[], int low, int high, int &comparisons) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        comparisons++;

        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high, int &comparisons) {
    if (low >= high)
        return;

    int split = partition(arr, low, high, comparisons);

    quickSort(arr, low, split - 1, comparisons);
    quickSort(arr, split + 1, high, comparisons);
}

void merge(int arr[], int low, int mid, int high, int &comparisons) {
    int leftSize = mid - low + 1;
    int rightSize = high - mid;

    for (int k = 0; k < leftSize; k++)
        mergeLeft[k] = arr[low + k];

    for (int k = 0; k < rightSize; k++)
        mergeRight[k] = arr[mid + 1 + k];

    int i = 0, j = 0, k = low;

    while (i < leftSize && j < rightSize) {
        comparisons++;

        if (mergeLeft[i] <= mergeRight[j])
            arr[k] = mergeLeft[i++];
        else
            arr[k] = mergeRight[j++];

        k++;
    }

    while (i < leftSize)
        arr[k++] = mergeLeft[i++];

    while (j < rightSize)
        arr[k++] = mergeRight[j++];
}

void mergeSort(int arr[], int low, int high, int &comparisons) {
    if (low >= high)
        return;

    int mid = (low + high) / 2;

```

```
    mergeSort(arr, low, mid, comparisons);
    mergeSort(arr, mid + 1, high, comparisons);
    merge(arr, low, mid, high, comparisons);
}

void siftDown(int arr[], int root, int size, int &comparisons) {
    while (true) {
        int largest = root;
        int left = 2 * root + 1;
        int right = 2 * root + 2;

        if (left < size) {
            comparisons++;

            if (arr[left] > arr[largest])
                largest = left;
        }

        if (right < size) {
            comparisons++;

            if (arr[right] > arr[largest])
                largest = right;
        }

        if (largest == root)
            return;

        int temp = arr[root];
        arr[root] = arr[largest];
        arr[largest] = temp;

        root = largest;
    }
}

void heapSort(int arr[], int n, int &comparisons) {
    comparisons = 0;

    for (int i = n / 2 - 1; i >= 0; i--)
        siftDown(arr, i, n, comparisons);

    for (int end = n - 1; end >= 1; end--) {
        int temp = arr[0];
        arr[0] = arr[end];
        arr[end] = temp;

        siftDown(arr, 0, end, comparisons);
    }
}

// Runs one algorithm on work[0..n-1], which the caller has already filled
int runSort(int which, int work[], int n) {
    int c = 0;

    if (which == 0) bubbleSort(work, n, c);
    if (which == 1) insertionSort(work, n, c);
    if (which == 2) selectionSort(work, n, c);
    if (which == 3) quickSort(work, 0, n - 1, c);
    if (which == 4) mergeSort(work, 0, n - 1, c);
    if (which == 5) heapSort(work, n, c);

    return c;
}

// ---- printing helpers, so that main() holds no loops ----

void fillRandom(int big[], int m) {
    srand(1);

    for (int k = 0; k < m; k++)
        big[k] = rand() % 100000 + 1;
}

void printCountTable(const char *names[], const int A[], const int up[],
                     const int down[], int work[], int n) {
    for (int s = 0; s < 6; s++) {
        cout << names[s] << "\t";

        if (s == 0 || s >= 3)
            cout << "\t";

```

```
        for (int k = 0; k < n; k++) work[k] = A[k];
        cout << runSort(s, work, n) << "\t";

        for (int k = 0; k < n; k++) work[k] = up[k];
        cout << runSort(s, work, n) << "\t";

        for (int k = 0; k < n; k++) work[k] = down[k];
        cout << runSort(s, work, n) << endl;
    }
}

void measureTimes(int big[], int work[], const int sizes[],
                  long long micros[][3]) {
    for (int t = 0; t < 3; t++) {
        int m = sizes[t];

        fillRandom(big, m);

        for (int s = 0; s < 6; s++) {
            for (int k = 0; k < m; k++)
                work[k] = big[k];

            auto t0 = high_resolution_clock::now();
            runSort(s, work, m);
            auto t1 = high_resolution_clock::now();

            micros[s][t] = duration_cast<microseconds>(t1 - t0).count();
        }
    }
}

void printTimeTable(const char *names[], long long micros[][3]) {
    for (int s = 0; s < 6; s++) {
        cout << names[s] << "\t";

        if (s == 0 || s >= 3)
            cout << "\t";

        cout << micros[s][0] << "\t" << micros[s][1] << "\t"
             << micros[s][2] << endl;
    }
}

void printFirstTen(const char *names[], const int big[], int work[]) {
    for (int s = 0; s < 6; s++) {
        for (int k = 0; k < 10000; k++)
            work[k] = big[k];

        runSort(s, work, 10000);

        cout << names[s] << ":\t";

        if (s == 0 || s >= 3)
            cout << "\t";

        printArray(work, 10);
    }
}

int main() {
    const char *names[6] = {"Bubble", "Insertion", "Selection",
                            "Quick", "Merge", "Heap"};

    static int big[10000];
    static int work[10000];

    // ---- Part 1: comparisons on the eight values of A ----
    int A[]    = {22, 6, 4, 32, 7, 9, 12, 5};
    int up[]   = {4, 5, 6, 7, 9, 12, 22, 32};
    int down[] = {32, 22, 12, 9, 7, 6, 5, 4};
    int n = 8;

    cout << "=== Part 1: comparisons on the eight values of A ===" << endl;
    cout << "Algorithm\tA\tsorted\treversed" << endl;
    printCountTable(names, A, up, down, work, n);

    // ---- Part 2: time on random arrays ----
    int sizes[3] = {1000, 5000, 10000};
    long long micros[6][3];

    measureTimes(big, work, sizes, micros);

```

```
    cout << endl << "=== Part 2: time on random arrays (microseconds) ===" << endl;
    cout << "Algorithm\tn=1000\tn=5000\tn=10000" << endl;
    printTimeTable(names, micros);

    // ---- Part 3: first 10 of each sorted array ----
    cout << endl << "=== Part 3: first 10 of each sorted array (n = 10000) ==="
         << endl;

    fillRandom(big, 10000);
    printFirstTen(names, big, work);

    return 0;
}
```

9 Execution screenshot

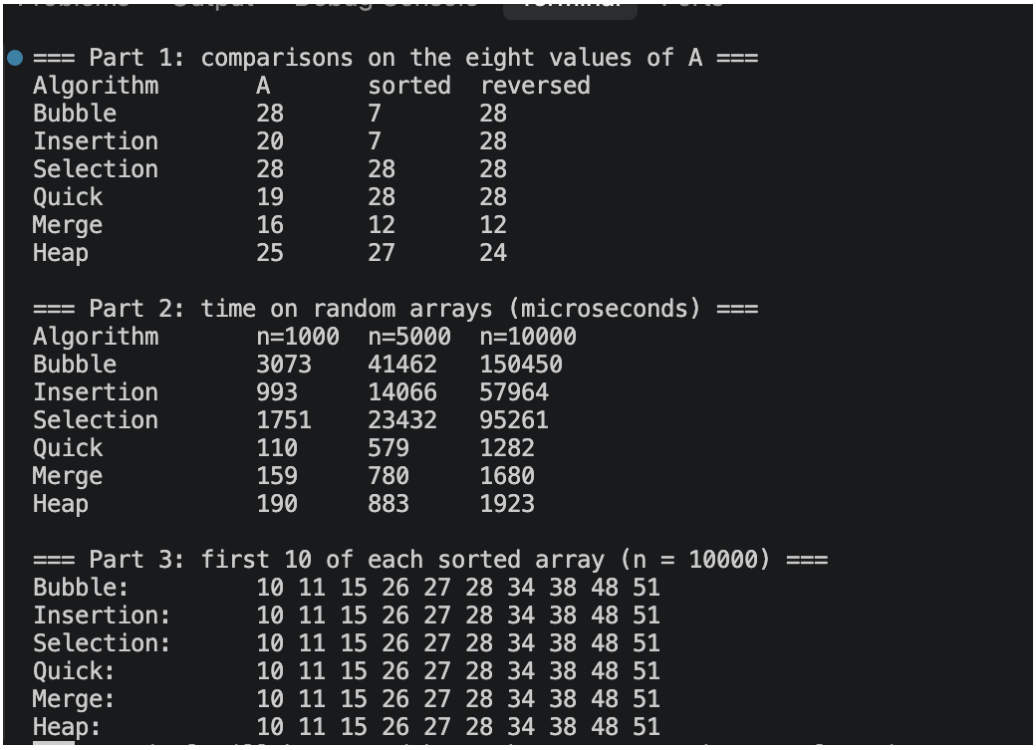


Figure 7: Program output: comparison counts, running times, and the correctness check.